3 unit

Java script

It is a high level light weigth programming language are to make webpages interactive and dynamic.

perpose of Java script.

-> To adding interactivity.
- Ex:- Js can respon to actions such as clicking on a botton
-> Form validation
-> Changing webpage content.
-> performing calculations.
-> handling events
- Ex:- Mouse click, press key, mouse mov.
-> Creating dynamic effect such as slideshow, popup, dropdown menu, animation.
-> comunication with the server

Java script programs structure.

Js can be included in a html document in 4 ways.

-> Java script in head section
-> Java script in body section
-> Java script in head and body section
-> Java script in external file

Java script in head section.

It is write inside the head tag

Syntax:

```
<html>
<head>
<script> //js code
</script>
</head>
<body>
</body>
</html>
```

perpose.

-> Js is commonly placed in head when defining a function that will be used by webpage.

Ex:-
```
<!doctype html>
<html>
<head>
<script>
function welcome(){
alert("Hi! welcome");
}
</script>
</head>
<body>
<button onclick="welcome()"> Click there</button>
</body>
</html>
```

Java script in body section.

It is write inside the body. tags

Syntax:-
```
<html>
<head>
</head>
<body>
<Script> Js code.
```

```
</script>
</body>
</html>
```

perpose: Js can be placed in body when code is related to content being displayed. or when we want html content to load before that script - exicute.

Ex:-
```
<!doctype html>
<html>
<head>
<title> Java script in body </title>
</head>
<body>
<h1> welcome to my website </h1>
<Script>
    document.write ("welcome");
</Script>
</body>
</html>
```

Java script in head and body section

It is write in both the body and head.

Syntax.
```
<html>
<head>
<script>
</script>
</head>
<body>
<Script>
</Script>
</body>
</html>
```

Ex:-
```
<!doctype html>
<html>
<head>
<script>
function greet()
{ alert ("Hello"); }
</script>
</head>
<body>
<h1> welcome </h1>
<button onclick = "greet()"> click me </button>
<Script > document.write ("Java script is running");
</script>
</body>
</html>
```

Java script in external file

It is write in another file and link the file in html

syntax

In html

```
< script src = "filename.js"></ script>
```

Ex:-
```
<!doctype html>
<html>
<head>
<script src ="script.js"></script>
</head>
<body>
```

```
<h1> welcome </h1>
<button onclick = "greet()"> click me </button>
</body>
</html>
```

```
Script.Js
function greet()
{ alert ("Hello"); }
```

Keywords in Javascript.

These are the reserved words that have pre define meanings in language

→ This cannot be used as a variable name, in function, identifiers

Keywords	discripstion
for	Creates a loop
function	It is used to define a function.
if	Executes a code based on condition
try	Defines a block for exception handling
catch	handles exception
class	declears a class
const	declears a constant
Continue	skips the current loop i'
else	It is an alternative for if statement
extents	indicates the class inheritance
Super	indicates the super class
while	Creates a looping statement.

Identifier in Javascript.

They are the names given to the variables, function objects, classes and other element in a javascript program

Rules for can define identifiers.

→ can contains letters, digits and underscore and dollers.

→ Cannot start with a digits

→ Cannot contain spaces

→ cannot be a javascript keyword

→ Javascript is a case sensitive

Valid	Unvalid
let studentname = x;	let 12a = s;

literales

This are the fixed values directly given in a javascript program

literals	Example	meaning
Boolean	true, false	logical value
Null	null	represents no value
Undefined	undefined	when value is not assigned.
String	"Hello". "123"	texted value
number	15, 30, 64, 5.5	numarical value
Array	[10, 20, 30]	collection of values
object	{ "name": "Anu", age: 19 }	collection of keyvalue pair
Bigint	9667778899711	large integer.

initial called as Live script
Mocha

Note:
literal is a value written on directly in sorcecode

Variable and it's types in JS
It is a named storage location used to store data values in a program
3 types of variables
- Var
- let
- Const.

Var
It is the older way of declering the variable
→ It can change the value and can be redeclared
Ex: var age = 20;
age = 21;
var age = 25;

let
Morden way of declering a variable
→ It can change the value but can not be redeclared
Ex: let age = 20;
age = 25;

Const
Used to declare a variable whose value should not be reassigned.
Ex: const a = 5;

Output statements
It is used to display or show the information to the user

① alert ()
Display the output in a popup massage box
Ex: alert ("Hello");

② document.write ( )
used to display output directly on the webpage
Ex:- document.write ("Hello world");
let age = 20;
document.write ("age = " + age)
let a = 10;
let b = 20;
document.write (a+b); 30
let name: "chandu";
let age = 19;
let city: "Huvina hadagali";
document.write (name + " " + age + " " + city);
document.write ("Name:" + name + " " + "Age:" + age + " " + "City:" + city);

③ Console.log ()
Use to display the output on the browser
Ex: let name = "Ammu";
console.log (name);

Operations & Operators in JS

It is a symbol used perform operation on values or variable

1) Arithmatic operator

Use to perform mathamatical operation

Write a program to add 2 no

```
<!doctype html
<html>
<head>
<title> operations </title>
</head>
<body>
<script>
let a=10
let b=20
document.write("sum:4.");
document.write(a+b);
</script>
</body>
</html>
```

+ -> addition
- -> subtraction
* -> multiplication
/ -> division
** exponual 5^10
% -> modlus
++ -> increment
-- -> decrement
x+ -> exponstal

```
<script>
let a=10;
let b=20;
document.write("sum:"+(a+b));
</script>
```

Assignment operator

It is used to assign or update the values in a variable

operator	Example	equivalent
=	a=5;	a=5
+=	let a+=5;	a=a+5
-=	let a-=5;	a=a-5
*=	let a*=5;	a=a*5
/=	let a/=5;	a=a/5
%=	let a%=5;	a=a%5
**=	let a**=5;	a=a**5

```
let x=20;
x/=4;          20/4
console.log(x);   = 5
```

Comparision operator

Used to compare two values

Operators	meaning
==	equal value.
!=	not equal to
>	greater than
<	lesser than
>=	greater than or equal
<=	lesser than or equal

Ex: let a = 5;
let b = 10;
.Consol.log (a==b);
output = false.

(-1 2-2 5-1 10-5

Logical operator

It mainly used to combain multiple combinations.

operator	meaning	Example
&&	and	console.log(10>5 && 20>10); true
||	OR	Console.log(10>5 || 20<10); true
!	NOT	let estudent=true; Console.log(!e-student);

String operator

The + operator is used to string concatination

Ex:- let name = "Chandu";
let age = "19";
Console.log("My name is" + name);
Console.log("I am" + age + "years old");

let massage = "Hello";
massage += "world";
Console.log(massage);

datatypes in javascript.

datatypes describes what kind of data or a value a variable contents of

There are 2 types

1. primitive datatype
2. Non-primitive datatype

questions

-> Explain client-server architecture. - (2) (static)
-> list client side & sower side technologies
-> Explain html table with an example
-> Explain the basic html page structure with an example
-> Explain the text formating html with example
-> Explain html list with example.
-> Explain the type CSS with syntax and example
-> Explain selectors with syntax & ample.
-> Explains css background with example..
-> Explain css short hand properties with example
-> Explain responsive design.

second internal

-> Explain html forms and its inputs element. - lab 10 5
-> Explain the conditional statement in JS with example
-> Explain java script (JS) uses with example
-> Explain array methods & string methods in javascript with example

Datatype	example
String	"Hello"
Number	25, 25.0, 25:12
boolean	True, false
Null	let x = null; intentionally empty
undefined	let x ; value as not empty been
BigInt	let x = 9G7116789 assigned

Non primitive datatype

The main non-primitive datatype is object. array and function also called as object

Ex: for primitive datatype

```
let name = "Sinchana";
let age = 19;
let bigint = 8431167781;
let isstudent: true;
let city ;
let reuslt = Null;
console.log (Name, typeof name);
console.log (age);
console.log (bigint);
console.log (isstudent);
console.log (city);
console.log (result);
```

Ex: for non-primitive datatype

```
let person = { name: "john", age: "25" };
let colors = ["red", "blue", "pink"];
function greet() { console.log ("Hello"); }
console.log (typeof person);
console.log (typeof colors);
console.log (typeof greet);
```

Type conversion:

Type conversion in javascript is process of converting a value from one datatype to another.

2 types.

① implicit type conversion
• Explicit type conversion

Explicit

It is type of conversion in which programmer manually converts datatype using functions such as · number() string() and boolean.

Ex:- let a = "100";
console.log(a, type of a);
a = number(a);
console.log(a, type of a);
output: 100 number.

Ex:- let x = "25.5";
console.log(number(x));
output: 25.5

let A=100;
console.log(a type of a);
a = String(a)
console.log(a, type of a);

let A=1;
A = boolean(a)
console.log(a, type of a)
output: true boolean.

Implicit type

It is the type of conversion means javascript automatically converts one data type to another without programmer manually converting it.

It is also called as automatic type conversion

String + number = string.

Ex:- let a = '10';
let b = 5;
let c = a+b;
console.log(c);
console.log(type of c, c) output: 105

string - number = number
let a = "10";
let b = 5;
let c = a-b;
console.log(c)
console.log(type of c, c) output = 5

string * number = number
let a = "10";
let b = 5;
let c = a*b;
Console.log(c);
console.log(type of c);
output = 50

string + number = number

```
let a = "10";
let b = 5;
let c = a/b;
console.log(c);
console.log(typeof c);
```

Taking input from the user

syntax:

```
let name = prompt("Enter your name:");
let age = prompt("Enter your age:");
console.log(age, type of age);
```

Note: prompt is a built in javascript function use to take input from the user

The value retured by prompt is string

Ex:- let age = Number(prompt("enter the age"));

Conditional statements

Conditional statements are used to decision making

Types

1) If
2) If-else
3) If-else-if
4) switch

If statement: executes only if the condition is true

syntax:

```
if(condition)
{ statement 1;
  statement n;
}
```

Ex:- let age = 20;

```
if(age >18)
{
console.log("Eligible to vote");
}
```

2. if-else statement - executes one block if the condition is true otherwise another block

syntax:

```
if (condition)
{
  statement 1;
  statement n;
}
else
{ statements; }
```

ex:- let age = 16;

```
if (age >18)
{
console.log("Eligible to vote");
}
else
{ console.log("Not eligible to vote");
}
```

3. If-else-if-else - used when there are multiple conditions.

syntax:

```
if(condition)
{ statements }
else if (condition)
{ statements }
```

```
else if (condition)
{ statement }
else
{ statements }
```

```
let marks = 75;
if (marks >= 90)
{ console.log ("Grade A+"); }
else if (marks >= 60)
{ console.log ("Grade A"); }
else
{ console.log ("Grade B"); }
```

o/p = A

4) switch statement:-

switch statement is a condition statement that allows program to select and execute one blocks of code from multiple case based on values of an expression

syntax:-

```
switch (expression)
{ case value1 : statements;
                break;
  case value2 : statements;
                break;
  ;
  ;
  default : statements ;
}
```

ex:-
```
let day = 2;
switch (day)
{ case 1 : console.log ("Monday");
           break;
  case 2 : console.log ("Tuesday");
           breaks;
  case 3 : console.log ("wednesday");
           break;
  default : console.log ("invalid day");
}
```

o/p = Tuesday.

case:- case is a keyword specifies a possible value.
breaks:- stop the execution of the switch.
default:- If is a keyword executes when no case matches.

Greatest of three numbers:-

```
let a=10;
let b=20;
let c=30;
if (a>b && a>c)
{ console.log ("Biggest no is:"+a); }
else if (b>a && b>c)
{ console.log ("Biggest no is:"+b); }
else
{ console.log ("Biggest no is:"+c);
}
let a = number (prompt ("enter the value of a"));
let b = number (prompt ("enter the value of b"));
let c = number (prompt ("enter the value of c"));
```

Looping statements

It is used to execute a block of code repeatedly as long as condition is true

Types
1) for
2) while
3) do while
4) for in
5) for of

for loop

It is used when number of repeations are known

syntax:- for(initialisation; condition; inc/dec)
{ //statement
}

Ex:- for(i=0; i<=5; i++)
{ console.log(i);
}

while loop

It is executes a block of code as long as condition is true.

syntax:- while(condition)
let i=0; { //statement; i++ }

Ex:- while(i<=5)
{ console.log(i);
i++;
}

do while loop

It is execute a block of code at least one because condition is check after loop body

syntax: do{
statements;
}
while(condition);

Ex:- let i=0;
do {
console.log(i);
i++;
}
while(i<=5);

for in loop

It mainly used to properties or keys of an object.

syntax:- for (let variable in object)
{ statements; }

Ex:- let student = { Name: "A";
Aage = 20;
city="xyz";
};

for(let i in student)
{ console.log(i, student[i]) }

name A
age 20
city xyz

for of loop

It is used with array or string

Syntax:- for (let variable of Array/string)
{ statement; }

used to iterate

Ex- let fruits = ["Apple", "Kiwi", "Strabarry"];
for (let f of fruits)
{ console.log(f);
}

Apple
Kiwi
strabarry

Write a Javascript program that reads a number using prompt and calculate its factorial

→ let n = Number(prompt("enter the value"));
let fact = 1;
for (i = 1; i <= n; i++)
{ fact = fact * i; 1×2 | 2×3 | 6×4 | 24×5 | 120
}
console.log("factorial of ", n, " is ", fact);
factorial of 5 is 120

Strings in javascript

It is a sequence of character, enclosed in single or double codes.

Ex:- "Hello", "123"

Creating string in JS

syntax:

let variable/string_name = "value";

Ex:- let a = "welcome@123";
console.log(a[3]);

Key characteristics of string.

→ It is a sequence of characters
→ Strings are indexed.

→ The length property ~~of~~ ~~strings~~ counterns number of character in a string,

Ex: console.log(a.length)

Program:

let a = prompt("enter the string");
console.log("The length of the string is", a.length);

→ Strings are immutable. That means indival. character
⇒ of string can't be change directly

Ex: let a = "Hello";
a[1] = "E"
console.log(a);
o/p = Hello.

String methods.

They are the operations that can be perform on string such as searching, extracting, modifying, replaceing or converting the string content.

Note:- there are built in methods

① modifying the case.

② toUpperCase() ③ toLowerCase()

Ex: let a = "Hello";
console.log(a.toUpperCase());
console.log(a.toLowerCase());

2) Extracting parts of the string

① slice() :- It is the method used to extract postion of a string

syntax: string_name.slice(start, end);

start → starting index (included) end → ending index (not included)

Ex: let str = "Hi How are you?";
console.log(str.slice(3,6));

④ substring() : extract the characteristic b/w indexes.
syntax: string.substring(start, end);
start - first index (included)
end - ending index (not included)
Ex: let str = "Javascript";
console.log(str.substring(4,10));

Searching and locating
① indexOf() :- returns the position of the first occurence of specified value
syntax: string.indexOf(value);
Ex: let a = "I am indian";
console.log(a.indexOf(I)); =0
"This is a apples Trees It grows apple";

② includes() :- It is a method to check whether a string contains specified values.
Ex: let str = "Hello world";
console.log(str, includes("world"));

triming void spaces
removing unwanted void spaces from a string.
Ex: let a = " Hellow world ";
console.log(a.trim());
O/P = Hellow world

trimstart()
control remove spaces only from the beinging

trimEnd()
remove spaces only from the end.

Character acesses method
They are is used to acesses the particuler character from a string using it's index.
① charAt()
returns the character at the specify index.
Ex: let a = "Hello";
console.log(charAt(2)); = l

② at() :- returns the character at the specified index.
Ex:- let a = "Hello";
conde.log(at(2));

Arrays in javascript.
It is the collection of multiple values stored in a single variable.
-> Values are stored in ordered maner and each value as index number.
~ It is not homogenoes.
Creating array.
It is created using squre brakets.
Syntax:
let arrayname = [value1, value2, valuen];
Ex: let fruits = ["Apple", "Kiwi", "orange"];
console.log("fruits");
let marks = [90, 70, 60, 75];
console.log("marks");

length.

length returns the number of elements in an array.

syntax: arrayname.length.

push()

It adds one or more elements to ~~end~~ the end of the array.

syntax: arrayname.push(element);

Ex: let names = ["Chowri", "Anu", "varun"];

names.push("sonu");

Conde.log(names);

pop()

It is used to remove the last element from an array.

syntax: arrayname.pop();

Ex: let marks = [1, 2, 3];

~~ma~~ marks.pop();

console.log(marks);

shift()

It removes the first element from an array

syntax: arrayname.shift();

ex- let marks = [1, 2, 3];

marks.shift();

console.log(marks);

unshift()

It used to add one or more elements at the beinging of the array.

syntax:- arrayname.unshift(element);

Ex:- let marks = [2, 3, 4, 5];

marks.unshift(1);

Conde.log(~~name~~ marks);

Map()

It is used to perform an operation on every element of an array and create a new array with result

syntax:- arrayname.map(function(element){

// operation

});

Ex:- suppose we wants to multiplay every number with 2:

let numbers = [1, 2, 3, 4];

let result = numbers.map(function(num){

return num * 2;

});

console.log(result);

Note: map method doesn't change the original array but it returns the new array.

filter()

It is used to select the elements that ~~stat~~ statifies the specified condition ~~and~~ & returns them as new array

Syntax:- arrayname.filter(function(element){

//operation

});

find num who greater than 10

Ex:- let numbers = [5, 12, 8, 20, 3];
let result = numbers.filter(function(num){
return num>10;
});
console.log(result);

reduce()
It is used to reduce all the elements of an array into single value.
→ It is commonly used for sum, product, total, average, minimum, maximum
syntax:- arrayname.reduce(function(accumulator, currentvalue){
return result;
}, initial value);
Ex:- let numbers = [10,20,30,40];
let total = numbers.reduce(function(sum, num){
return sum + num;
}, 0);
console.log(total);

find even
let numbers = [1,2,3,4];
let result = number.filter(function(num){
return num%2==0;
});
console.log(result);

indexOf()
It find the index position of the specified element in an array.
syntax: arrayname.indexOf(element);
Ex:- let fruits = ["Apple", "Grapes", "Mango"];
console.log(fruits.indexOf("Grapes"));

5) write js program that reduce numbers and stores it in an array & search the given number in the stored array using linear search.

```
<!doctype html>
<html>
<head>
<title> Linear search </title>
</head>
<body>
<script>
let a=[];
let n=5;
for(let i=0; i<5; i++)
{
a[i] = number(prompt("enter number"));
}
let key = number(prompt("enter key to search"));
let found = false;
for(let i=0; i<n; i++)
{ if(a[i] == key)
{ found = true;
break;
}
}
```

```
if (found)
    document.write ("Number is found".i);
else
    document.write ("Number is not found");
</script>
</body>
</HTML>
```

function in java script

Block of reusable code design to perform a specific task

syntax: function functionname(parameters)
{
statements;
}

→ keyword → parameters

Example: function addition(a,b)
{ let sum = a+b; // function body
console.log(sum);
}
addition(5,6); → arguments

function parameters

parameters are the variables & written inside the parantheses of a function definition
→ parameters receives the inputs when the function is called.

ex:
~~greet~~ function greet(name, age)
{ console.log("name is", name);
console.log("age is", age);
}
greet(Ammu, 20);

arguments

It is the actual value passed during function call.

function ~~invocation~~ invocation.

It mean executing function by calling it.

function call()

If a function can invokate using call() method
-> It allows to specifies the value of "this" explicitly.
-> arguments are passed ~~individually~~ individually.
syntax: functionname.call(thisValue, arg1, arg)

this: It is a keyword refers to object that is associated with calling the current function.
-> It's value depends on how the function is called.
Ex:- let student = {name: "sonu"};
function greet(age){
console.log("Name", this.name);
console.log("Age:", age);
}
greet.apply(student, [27]);
Ex:- let student = {name: "A"};
function greet(age){
console.log("Name is:", this.name);
console.log("Age is:", age); }
greet.call(student, 27);

→ Call() is useful in we want a function execute with particular object using this,

apply() invocate

It invocates a function with specified this value But arguments are passed as array

syntax:

functionname.apply (thisvalue, [argument, argument2]);

Ex: let student = { name: "A" };

function greet (age){

console.log ("Name", this.name);

console.log ("age", age);

}

greet.apply (student, [27, 98]);

Ex let A = { name: "chandana" };

function greet (age, city)

{ console.log ("name:", this.name);

console.log ("age:", age);

console.log ("city:", city);

}

greet.apply (A, [20, "Svg"]);

bind()

It creates a new a function with specified this value and optionaly predefine argument.

Note: It doesn't indiately execute the function

syntax:

let new function = function.bind (thisvalue, argument);

Ex let new = { n: "A" };

function greet (age, city) {

console.log ("Name:", this);

console.log ("Age:", age);

console.log ("city:", city);

}

let newgreet = greet.bind (student, 20, "14V Hall");

newgreet ();

method	execute indidately?	arrayements
call()	Yes	individual
apply()	Yes	array
bind()	No we have to new function explicitly	individual

closers closures()

It is created when inner function remember and can access a variable of its another function, even after outer function as finished executing.

syntax:

function outerfunction () {

let variable = value;

function innerfunction () {

// access variable

}

return innerfunction;

}

let returns = outerfunction ();

returns();

Ex: function counter () {
let count = 0;
function increase () {
count++;
console.log (count);
}
return increase;
}
let c = counter ();
c();

1

CSS frame work
CSS frame work is a collection of prereturn CSS styles and components used to design and develop webpages quickly and easily.

main uses:
- Responsive webdesign
- Faster development
- Readymade components (such as buttons, tables, Forms)
- Easy layout design
- Cross browser compatibility
- Easy maintainance
- consistence design (same in all device)

Example
→ Bootstrap
→ Foundation
→ Tailwind CSS
→ Bulma

Bootstrap (we are using bootstrap 5)
Is a popular CSS framework used to create responsive and attractive website.

features
- Responsive design.
- Mobile first approach - It is design design starting with a mobile devices and then supports larges screams.
- Ready made components (Button, table, form, cards, alerts)
- Utility class - Bootstrap provides readymade classes for common styling task
 Hello
 content
M - margin P2 - padding
T - top

* Javascript components
1) Drop down
2 sliders
3 tabs
4 side navigation
5 icon supports
6 mobile friendly
7 Easy to use
8 customisable - layouts components can be customise according to the requirement.
9) CSS helper classes (alignment, colors, spacing, text formatting)

w. grid system - It provides 12 column responsive grid system for creating webpage layout.

Materialize Css

Is a modern css frame work based on material design principle develop by google.

- material design based - providing clean, modern, user friendly interface.

* drop down
* sliders
* tabs
* side navigation
* icon supports
* mobile friendly
* Easy to use
* customisable
* Css helper classes
* grid system

Html form

It is Html element we to collect input from the user and send it to the server for processing

syntaxL.

```
<form action = "submit.php" method = "post">
        form elements
</form>
```

<form> : this tag is to be use when form as to created

action = "submit.php" :- specifies where the form data is send after submition.

Method = "post" / "get"

Specifies how the data is sent to the server.

Note:

post method doesnot display the data in the url.

Basic Html form.

```
<label> Name: </label>
<input type = "text" name = "student_name">
<br> <br>
<label> password: </label>
<input type = "password" name = "password">
<br> <br>
<label> emile: </label>
<input type = "emile" name = "student_emile">
<br> <br>
<label> gender: </label>
<input type = "radio" name = "student_gender" value = "male">
male
<input type = "radio" name = "student_gender" value = "female">
female
<br><br>
<label> course: </label>
<select name = "course">
<option> BCA </option>
<option> BA </option>
<option> B.com </option>
</select>
<br><br>
<label> address: </label>
<textarea name = "student_address"></textarea>
<br> <br>
<input type = "submite" value = "submit">
</form>
```

textarea

It is the element used to except multi line text input. from the user.

placeholder

It displays the inte before typing